

SQL injection, System rejestracji

Wykłady 7

Bartosz Lauks

21 kwietnia 2026



- 1 SQL injection
- 2 System rejestracji
- 3 Haszowanie haseł
- 4 Dobre praktyki bezpieczeństwa

- 1 SQL injection
- 2 System rejestracji
- 3 Haszowanie haseł
- 4 Dobre praktyki bezpieczeństwa

SQL injection

Czym jest SQL injection ?

- **SQL Injection** (wstrzykiwanie SQL) to jedna z najstarszych i najgroźniejszych podatności aplikacji internetowych.
- Polegająca na manipulacji zapytaniami SQL poprzez wprowadzenie złośliwego kodu do pól wejściowych aplikacji (np. formularzy, parametrów w adresie URL, nagłówków HTTP).
- Celem ataku jest zmiana treści zapytania SQL wykonywanego przez aplikację w taki sposób, by uzyskać nieautoryzowany dostęp do danych, zmodyfikować je, usunąć lub nawet przejąć kontrolę nad całym systemem bazodanowym.

SQL injection

Skutki SQL injection

- **Nieautoryzowany dostęp do danych** – atakujący może odczytać poufne informacje (np. loginy, hasła, dane osobowe użytkowników).
- **Modyfikacja danych** – możliwość zmiany zawartości bazy danych (np. zmiana hasła administratora, edycja rekordów).
- **Usunięcie danych** – atakujący może usunąć rekordy lub całe tabele (np. **DROP TABLE**).
- **Ominięcie mechanizmów uwierzytelniania** – zalogowanie się bez znajomości poprawnych danych (np. jako administrator).
- **Przejęcie kontroli nad aplikacją** – w niektórych przypadkach możliwość wykonania operacji administracyjnych.
- **Ujawnienie struktury bazy danych** – błędy SQL mogą zdradzić strukturę tabel i zapytań.

SQL injection

SQL injection przykłady

- Niestety, ale w obecnej formie nasza aplikacja jest podatna na taki rodzaj ataków.
- Przejdź do formularza logowania i podaj takie dane:
 - **Nazwa użytkownika** - dowolne słowo
 - **password** - " OR 1=1 – "(bez ")
 - **Nazwa użytkownika** - admin" – "
 - **password** - hasło dowolne
- Do systemu można dodać odpowiednią klauzulę, aby wyświetlić dane wszystkich userów lub usunąć całą tablicę **user**.
 - **Zmiana hasła** - ' UNION SELECT null, 'admin', '1234' –
 - **Wyświetl dane z tablicy** - ' UNION SELECT null, name, password FROM user –
 - **Usuń tablice** - '; DROP TABLE user; –

SQL injection

Obrona przed SQL injection

- Jedną z podstawowych form obrony przed tą podatnością jest walidacja danych.
- Dane w formularzach muszą zostać odpowiednio przefiltrowane i sprawdzone przed wysłaniem ich na serwer.
- W wyżej wymienionych przykładach wstrzykiwanie SQL można zauważyć, że do każdego sposób włamania jest wymagany przynajmniej jednej symbol (' = - itd.)

SQL injection

Obrona przed SQL injection

- Jest to często praktyka w serwisach internetowych, gdzie formularze zwracają nam błąd gdy w polach znajduje się jakiśkolwiek symbol.
- Do zaimplementowania takiego mechanizmu w **HTML** wystarczy dodać atrybut **pattern**, który określa wyrażenie regularne, z którym sprawdzana jest wartość pola wejściowego w momencie przesyłania formularza.

Przykład : localhost/lecture7/index.php

SQL injection

Obrona SQL injection po stronie serwera

- Niestety takie zabezpieczenie jest niewystarczające.
- Użytkownik może posiadać podstawową wiedzę o stronach WWW i edytować kod HTML w celu usunięcia zabezpieczeń.
- Każda walidacja powinna zostać wykonana po stronie **klient** jak i **serwera**.
- Walidacja w przeglądarce powinna odfiltrowywać nasz system od niepotrzebnych żądań wysyłanych na serwer, które na pewno zwrócą błąd.

Przykład : localhost/lecture7/login.php

SQL injection

Obrona SQL injection a mysqli

- Niewłaściwe użycie **mysqli** (np. konkatencja stringów w zapytaniach SQL) może prowadzić do podatności SQL Injection”
- Zaleca się rezygnację z tej klasy na rzecz znacznie bezpieczniejszego i wygodniejszego rozszerzenia **PDO**.

Co to jest PDO ?

- **PDO (PHP Data Objects)** to interfejs warstwy dostępu do bazy danych w PHP, który zapewnia jednolity sposób komunikacji z wieloma różnymi systemami bazodanowymi.
- **PDO** nie jest konkretnym sterownikiem bazy danych, lecz abstrakcyjnym interfejsem, dzięki któremu można pisać kod niezależny od konkretnego silnika baz danych.

Zalety PDO

- **Niezależność od bazy danych** – zmiana bazy danych często wymaga tylko zmiany DSN (Data Source Name), a nie całego kodu.
- **Wsparcie dla przygotowanych zapytań (prepared statements)** – co zwiększa bezpieczeństwo (ochrona przed SQL Injection).
- **Obsługa transakcji** – można łatwo rozpocząć, zatwierdzić lub cofnąć transakcję.
- **Zwięzły i czysty kod** – ułatwia czytelność i utrzymanie.
- **Obsługa błędów i wyjątków** – pozwala reagować na błędy w sposób obiektowy.

Przykłady użycia PDO : <localhost/lecture7/PDO.php>

Logowanie z PDO : <localhost/lecture7/loginPDO.php>

Zmiana w action : <localhost/lecture7/index.php>

- 1 SQL injection
- 2 System rejestracji
- 3 Haszowanie haseł
- 4 Dobre praktyki bezpieczeństwa

System rejestracji

Kroki systemu rejestracji użytkownika

- Użytkownik dostaje się przez hiperłącze do strony, gdzie musi podać dane do logowania. ([register.php](#))
- Użytkownik wpisuje dane do formularza i zatwierdza go.
- Walidacja po stornie klienta
- Przekierowanie formularza do pliku, gdzie dane zostaną sprawdzone. ([registerServer.php](#))
- Haszowanie hasła.
- Połączenie do bazy danych za pomocą PDO oraz wprowadzenie danych o nowym użytkowniku.
- Przekierowanie do formularza logowania z informacją zwrotną z serwera.

Formularz: localhost/lecture7/register.php

System rejestracji

Blokada dostępu dla zalogowanych użytkowników

- Warto zabezpieczyć stronę z formularzem rejestracji przed zalogowanymi użytkownikami.
- Dobrym sposobem będzie sprawdzenie na początku skryptu rejestracji ([register.php](#)) czy w **zmiennych sesyjnych** zawiera się zmienna **logged**, która informuje o statusie usera.
- Hiperłącza powinny być dynamiczne. Jeśli status użytkownika to "zarejestrowany" linki do podstrony rejestracji nie powinny się wyświetlać

System rejestracji

Walidacja po stronie klienta

- Strona z formularzem powinna być tak zaimplementowana, aby uniemożliwić wysłanie **request**, gdy dane nie są poprawne. Na przykład:
 - Pola są puste.
 - **Nazwa użytkownika** i **hasło** nie składają się z 3-20 znaków, tylko litery i cyfry.
 - **Hasło** oraz "**Potwierdź hasło**" nie są identyczne. Niestety nie ma atrybuty w HTML, który sprawdza to działanie.
- Jeśli dane są prawidłowe formularz zostaje wysłany na serwer ([registerServer.php](#))

Przykład: [localhost/lecture7/register.php](#)

System rejestracji

Tworzenie tablicy user

- Ponieważ modyfikujemy system tak, aby oprócz nazwy użytkownika i hasła przechowywał dodatkowo pole **email** - **VARCHAR(255)**.
- Musimy stworzyć nową tablicę **user** w naszej bazie danych lub zmodyfikować tę istniejącą.
- Dodajmy **INDEX UNIQUE** dla **email** oraz **name**, aby już na poziomie bazy danych nie dopuścić do pojawienia się duplikatów kont użytkownika.

System rejestracji

Wykorzystanie PDO do połączenia z bazą danych

- W pliku `dbPDO.php` zostaną zamieszczone dane służące do połączenia z bazą danych.
- Niżej utworzymy instancję połączenia z bazą danych.
- **PDO** w przeciwieństwie do **mysqli** jako argument przyjmuje **DSN** czyli łańcuch znaków, który zawiera informacje potrzebne do nawiązania połączenia z bazą danych.
- Próba połączenia musi zawierać się w bloku **try..catch**.
- W przypadku błędu połączenia z bazą danych w PHP, należy wypisać komunikat o błędzie i zakończyć działanie skryptu.

Przykład : <localhost/lecture7/dbPDO.php>

System rejestracji

Walidacja, czy użytkownik lub email już istnieje

- Połączenie z bazą danych można zaimportować za pomocą **require_once**, co umożliwia jego ponowne wykorzystanie bez ryzyka wielokrotnego załączenia.
- Sprawdzamy, czy konto z podanym w formularzu adresem e-mail i nazwą użytkownika istnieje w bazie danych.
- Jeśli tak zwracamy błąd i kierujemy się znów do formularza.

Zmiany : <localhost/lecture7/registerServer.php>

- 1 SQL injection
- 2 System rejestracji
- 3 Haszowanie haseł
- 4 Dobre praktyki bezpieczeństwa

Haszowanie haseł

Co to jest haszowanie haseł?

- **Haszowanie haseł** to proces jednokierunkowego przekształcania hasła (ciągu znaków) w ciąg o stałej długości (tzw. skrót, **hash**), który reprezentuje oryginalne hasło.
- Nie da się z niego w prosty sposób odzyskać pierwotnej treści. Dzięki temu hasło użytkownika może być bezpiecznie przechowywane w bazie danych.

Cechy funkcji haszujących

- **Jednokierunkowość** – nie da się odtworzyć hasła z hasza.
- **Odporność na kolizje** – trudno znaleźć dwa różne dane wejściowe dające ten sam hash.
- **Deterministyczność** – to samo wejście daje zawsze ten sam wynik.
- **Odporność na ataki** - słownikowe oraz tęczowymi tablicami

Haszowanie haseł

Algorytmy hasujące

- Do zmiany hasła w hash używa się odpowiednich do tego celu algorytmów.
- Algorytmy takie jak **MD5**, **SHA1**, czy nawet **SHA256** były tworzone z myślą o szybkości.
- Dzisiaj ataki z wykorzystaniem GPU mogą sprawdzić miliardy możliwych haseł na sekundę, co czyni je nieodpowiednimi do przechowywania haseł.
- Na potrzeby dzisiejszych standardów bezpieczeństwa w pełni odpowiada algorytm **bcrypt**.

Haszowanie haseł

haszowanie haseł w PHP

- Do haszowanie haseł w PHP używane się funkcji `password_hash($password, PASSWORD_DEFAULT)`.
- Zwraca hash na podstawie podanej wartości.
- Drugi argument jest opcjonalny. Za jego pomocą można zadeklarować jaki algorytm zostanie wykorzystany.
- Jeśli używany PHP jest w najnowszej wersji domyśle zostanie użyty najlepszy algorytm.

Haszowanie haseł

Zakończenie rejestracji

- Przetworzone przez funkcję hasło wraz z resztą danych zostanie przesłane do bazy danych.
- Po udanej rejestracji aplikacji prześnienie użytkownika do podstrony logowania.

Dodanie haszowania : <localhost/lecture7/registerServer.php>

Haszowanie haseł

Zmiany w systemie logowania

- Aby system logowania w pełni działał należy zmodyfikować działanie skryptu logowania.
- Zastąpić **mysqli** interfejsem **PDO** w ([loginPDO.php](#)).
- Pobrać dane użytkownika (jeśli taki istnieje) z bazy danych oraz porównać przesłane hasło z haszem przy pomocy funkcji **password_verify()**.
- Jeśli wszystko się zgadza użytkownik powinien zostać zalogowany.

[Dodanie haszowania : localhost/lecture7/loginPDO.php](#)

- 1 SQL injection
- 2 System rejestracji
- 3 Haszowanie haseł
- 4 Dobre praktyki bezpieczeństwa

Dobre praktyki bezpieczeństwa

OWASP Top 10

- **OWASP (Open Web Application Security Project)** to organizacja zajmująca się bezpieczeństwem aplikacji webowych.
- Publikuje listę **OWASP Top 10** – najczęstszych i najgroźniejszych podatności.
- **SQL Injection** należy do kategorii:
 - **A03: Injection** (r. 2025)
- Lista OWASP pomaga programistom:
 - identyfikować zagrożenia
 - stosować dobre praktyki
 - projektować bezpieczne aplikacje

Dobre praktyki bezpieczeństwa

Najważniejsze zalecenia OWASP

- **Stosowanie prepared statements** (PDO / mysqli)
- **Walidacja danych wejściowych**
 - po stronie klienta i serwera
- **Unikanie dynamicznego budowania zapytań SQL**
- **Obsługa błędów**
 - nie wyświetlać szczegółów błędów użytkownikowi
- **Aktualizacja oprogramowania**
 - biblioteki, frameworki, silnik bazy danych

Dobre praktyki bezpieczeństwa

Zasada najmniejszych uprawnień (Least Privilege)

- Każdy użytkownik systemu powinien mieć tylko takie uprawnienia, jakie są mu niezbędne do działania.
- Dotyczy to również użytkownika bazy danych używanego przez aplikację.
- Ograniczenie uprawnień minimalizuje skutki ewentualnego ataku.
- W konsoli [phpmyadmin](#) w sekcji [privileges](#) możemy stworzyć dedykowane konto wykorzystywane przez naszą aplikację (PHP), które będzie miało ograniczone uprawnienia.

Dobre praktyki bezpieczeństwa

Przykłady zastosowania Least Privilege

- Aplikacja nie powinna używać konta **root** do połączenia z bazą danych.
- Konto bazy danych powinno mieć tylko niezbędne uprawnienia:
 - **SELECT** – odczyt danych
 - **INSERT** – dodawanie danych
 - **UPDATE** – modyfikacja danych
- Należy unikać nadawania uprawnień:
 - **DROP, ALTER, DELETE** (jeśli nie są wymagane)
- W przypadku SQL Injection:
 - atakujący ma ograniczone możliwości działania

Zmiana danych user DB : <localhost/lecture7/dbPDO.php>

Dziękuję za uwagę !

Bartosz Lauks